Normalising Pipe-Network Measurement Data to 3NF

Step-by-step solution: from a flat CSV file to a relational schema in third normal form

1. Scenario and Starting Point

A water utility monitors a small pipe network. Pipes connect junctions (nodes). Sensors are mounted on pipes and each sensor records one or more physical quantities (pressure, flow rate, temperature). Every 10 minutes a value is logged per sensor and quantity. The logger exports everything into a single flat file **measurements_flat.csv** (36 rows, 16 columns), where every row repeats all information about the sensor, its pipe and the pipe's junctions.

Columns of the flat file:

#	Column	Example	Description
1	timestamp	2026-11-10 08:00:00	Time of the reading
2	sensor_code	S-101	Sensor identifier
3	sensor_name	Inlet probe	Human-readable sensor name
4	sensor_position_m	15	Mounting distance from pipe start (m)
5	pipe_code	P-01	Pipe identifier
6	pipe_material	PVC	Pipe material
7	pipe_diameter_mm	110	Nominal diameter (mm)
8	pipe_install_year	2008	Year of installation
9	from_junction	J-A	Junction where the pipe starts
10	from_junction_position	0;0	Position of start junction as "x;y" (m)
11	to_junction	J-B	Junction where the pipe ends
12	to_junction_position	120;0	Position of end junction as "x;y" (m)
13	quantity	pressure	Physical quantity measured
14	unit_symbol	bar	Unit symbol
15	unit_name	Bar	Full unit name
16	value	4.11	Measured value

Excerpt (first 6 rows, selected columns):

timestamp	sensor_code	pipe_code	pipe_material	from_junction	to_junction	unit_symbol	value
2026-11-10 08:00:00	S-101	P-01	PVC	J-A	J-B	bar	4.11
2026-11-10 08:00:00	S-101	P-01	PVC	J-A	J-B	degC	11.02
2026-11-10 08:00:00	S-102	P-01	PVC	J-A	J-B	m3/h	38.69
2026-11-10 08:00:00	S-201	P-02	Cast iron	J-B	J-C	bar	3.68
2026-11-10 08:00:00	S-301	P-03	PE-HD	J-C	J-D	bar	3.92
2026-11-10 08:00:00	S-301	P-03	PE-HD	J-C	J-D	m3/h	37.39

2. Step 0 – Analyse the Problems of the Flat Table

Before normalising, look for redundancy. In the flat file, the fact *"pipe P-01 is made of PVC, 110 mm, installed 2008"* is stored in 18 of the 36 rows. This redundancy causes the classic **anomalies**:

- **Update anomaly** – if P-01 is relined and its material changes, 18 rows must be changed. Forgetting one row produces contradictory data.
- **Insert anomaly** – a new junction or a newly installed sensor cannot be stored until it has produced at least one measurement (there is no row to put it in).
- **Delete anomaly** – deleting the old measurements of sensor S-201 also deletes the only record that pipe P-02 is cast iron from 1987.
- **Non-atomic values** – the junction positions are stored as text `"120;80"` (two values in one field).

3. Step 1 – First Normal Form (1NF)

Rule: every attribute holds exactly one atomic value, there are no repeating groups, and every row is identifiable by a primary key.

Actions:

- Split `from_junction_position` into `from_junction_x` and `from_junction_y` (same for `to_junction_position`). Now every field is atomic and you can calculate with the coordinates.
- Determine the primary key. A single reading is identified by **when** it was taken, **which sensor** took it and **which quantity/unit** it is: `PK = (timestamp, sensor_code, unit_symbol)`. `(timestamp, sensor_code)` alone is not enough, because sensor S-101 records pressure *and* temperature at the same moment.

Result: one table **R1** with 18 atomic columns and a composite key. The table is in 1NF, but all redundancy is still there.

4. Step 2 – Identify the Functional Dependencies

A functional dependency A → B means: if you know A, B is uniquely determined. Derive them from the meaning of the data (not only from the 36 sample rows!):

#	Dependency	Type relative to the key (timestamp, sensor_code, unit_symbol)
FD1	timestamp, sensor_code, unit_symbol → value	full dependency on the whole key – OK
FD2	sensor_code → sensor_name, sensor_position_m, pipe_code	**partial** – depends on only a part of the key
FD3	unit_symbol → unit_name, quantity	**partial** – depends on only a part of the key
FD4	pipe_code → pipe_material, pipe_diameter_mm, pipe_install_year, from_junction, to_junction	**transitive** – sensor_code → pipe_code → material …
FD5	junction → x, y (for both roles: from / to)	**transitive** – pipe_code → from_junction → x, y

Note: a sensor can measure several units and a unit is used by several sensors, so there is **no** dependency `sensor_code → unit_symbol` nor `unit_symbol → sensor_code`. This becomes important in Step 5.

5. Step 3 – Second Normal Form (2NF)

Rule: the table is in 1NF and every non-key attribute depends on the *whole* primary key (no partial dependencies). 2NF is only an issue for composite keys – which we have.

Actions: move each partial dependency (FD2, FD3) into its own table, with the determinant as the new primary key. The determinant stays in the original table as a foreign key so nothing is lost.

Table	Columns (PK underlined)	Comes from
measurements	timestamp, sensor_code, unit_symbol, value	FD1
sensors_2nf	sensor_code, sensor_name, sensor_position_m, pipe_code, pipe_material, pipe_diameter_mm, pipe_install_year, from_junction, from_x, from_y, to_junction, to_x, to_y	FD2 (+ everything that depends on it transitively)
measurement_units	unit_symbol, unit_name, quantity	FD3

Result: the 2NF tables. The unit and measurement tables are already fine, but **sensors_2nf** still repeats pipe and junction data for every sensor on the same pipe (S-101 and S-102 both carry “PVC, 110, 2008”).

6. Step 4 – Third Normal Form (3NF)

Rule: the table is in 2NF and no non-key attribute depends on another non-key attribute (no transitive dependencies: `Key` → `X` → `Y`).

Actions:

- FD4: `sensor_code` → `pipe_code` → `material, diameter, year` … – move the pipe attributes to a new table **pipes**. `sensors` keeps only `pipe_code` as foreign key.
- FD5: `pipe_code` → `from_junction` → `x, y` – move the coordinates to a new table **junctions**. `pipes` keeps two foreign keys: `from_junction` and `to_junction`. Because both columns describe the same kind of thing (a junction), only **one** junctions table is needed – the two FK columns are just different *roles*.

Table	Columns (PK underlined)
junctions	junction_code, x, y
pipes	pipe_code, material, diameter_mm, install_year, from_junction (FK), to_junction (FK)
sensors	sensor_code, sensor_name, position_m, pipe_code (FK)
measurement_units	unit_symbol, unit_name, quantity
measurements	timestamp, sensor_code, unit_symbol, value

Now every non-key attribute depends on *“the key, the whole key, and nothing but the key”*. The schema is in 3NF.

7. Step 5 – Resolve the Many-to-Many Relationship sensors ↔ units

The 3NF result above is correct, but one fact is still not stored on its own: **which quantities a sensor is able to measure**. S-101 measures pressure and temperature; pressure (bar) is measured by S-101, S-201 and S-301. That is an **m:n relationship**.

- A relational table cannot store an m:n relationship directly (a column `units = "bar, degC"` would break 1NF again).
- Solution: an **associative (junction) table sensor_unit_rel(sensor_id, unit_id)**. Its primary key is the combination of both foreign keys, so every pair exists only once.
- Benefit 1: a new sensor can be registered with its configured units *before* it sends any data (removes the insert anomaly).
- Benefit 2: **measurements** references `(sensor_id, unit_id)` in **sensor_unit_rel** with a *composite* foreign key. The database therefore rejects nonsense such as a temperature value from a pure flow meter.

8. Step 6 – Surrogate Keys and Final Schema

Natural keys such as `'P-01'` work, but in practice we add an integer **surrogate key** `id` to every entity table: it is short, never changes (even if a pipe is renamed) and makes joins fast. The natural code is kept with a `UNIQUE` constraint so duplicates are still impossible. The measurement key `(time, sensor_id, unit_id)` remains as a `UNIQUE` constraint. Surrogate keys do not change the normal form.

junctions

Column	Type	Key / constraint	Meaning
id	INTEGER	PK	Surrogate key
code	TEXT	NOT NULL, UNIQUE	e.g. J-A
x_m	REAL	NOT NULL	x coordinate (m)
y_m	REAL	NOT NULL	y coordinate (m)

pipes

Column	Type	Key / constraint	Meaning
id	INTEGER	PK	
code	TEXT	NOT NULL, UNIQUE	e.g. P-01
material	TEXT	NOT NULL	
diameter_mm	INTEGER	NOT NULL	
install_year	INTEGER	NOT NULL	
from_junction_id	INTEGER	FK → junctions.id	start node
to_junction_id	INTEGER	FK → junctions.id	end node (CHECK ≠ from)

sensors

Column	Type	Key / constraint	Meaning
id	INTEGER	PK	
code	TEXT	NOT NULL, UNIQUE	e.g. S-101
name	TEXT	NOT NULL	
pipe_id	INTEGER	FK → pipes.id	pipe the sensor is mounted on
position_m	REAL	NOT NULL	distance from start junction

measurement_units

Column	Type	Key / constraint	Meaning
id	INTEGER	PK	
symbol	TEXT	NOT NULL, UNIQUE	bar, m3/h, degC
name	TEXT	NOT NULL	full unit name
quantity	TEXT	NOT NULL	pressure, flow rate, temperature

sensor_unit_rel

Column	Type	Key / constraint	Meaning
sensor_id	INTEGER	PK, FK → sensors.id	
unit_id	INTEGER	PK, FK → measurement_units.id	

measurements

Column	Type	Key / constraint	Meaning
id	INTEGER	PK	
time	TIMESTAMP	NOT NULL	
sensor_id	INTEGER	FK (sensor_id, unit_id) → sensor_unit_rel	composite FK, part 1
unit_id	INTEGER	(same composite FK)	composite FK, part 2
value	REAL	NOT NULL	measured value
		UNIQUE (time, sensor_id, unit_id)	one value per sensor/unit/time

9. The Relationships – and Why

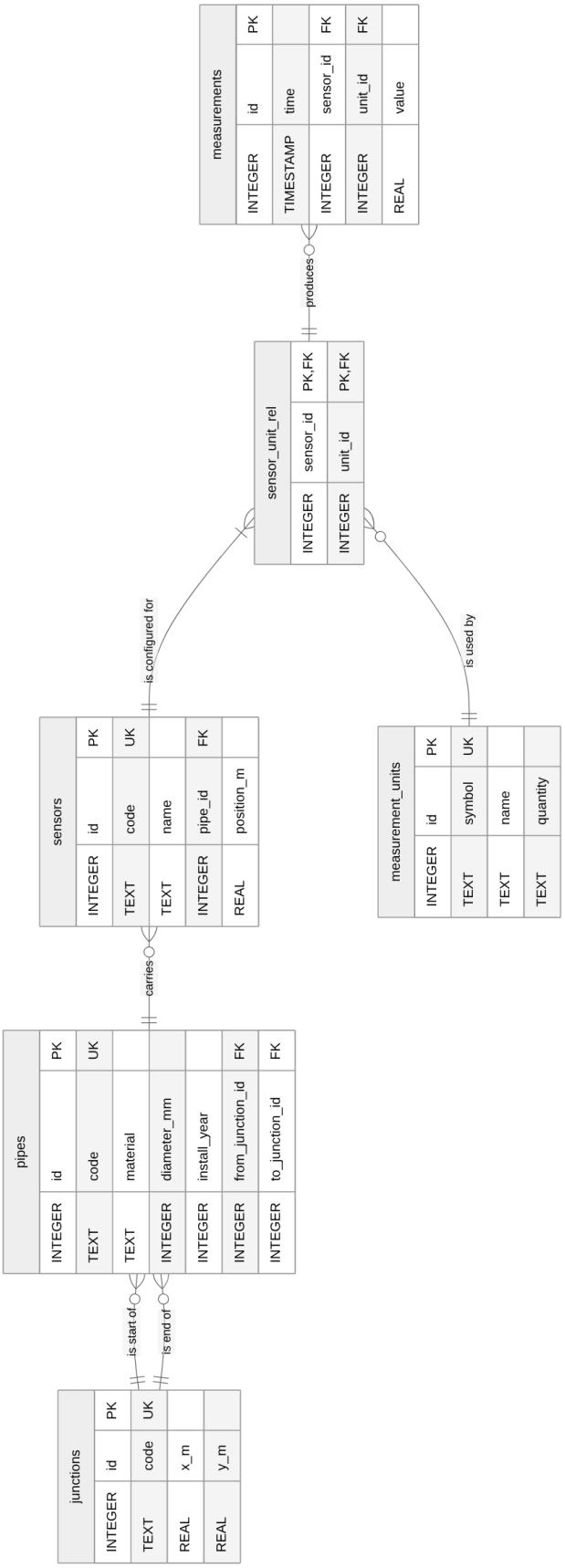
Cardinalities are read “one A has … B”. The foreign key is always placed on the **n** side.

Relationship	Card.	Foreign key	Why this relationship?
junctions → pipes (start)	1 : n	pipes.from_junction_id	A pipe starts at exactly one junction; many pipes can start at the same junction (a junction is a branching point). Mandatory: a pipe cannot exist without a start.
junctions → pipes (end)	1 : n	pipes.to_junction_id	Same reasoning for the end point. Two FKs to the same table = two different roles. Storing both ends makes the network a graph you can traverse with SQL.
pipes → sensors	1 : n	sensors.pipe_id	A sensor is physically mounted on exactly one pipe; a pipe can carry 0..n sensors. Pipe data is stored once instead of once per sensor (removes FD4 redundancy).
sensors ↔ measurement_units	m : n	via sensor_unit_rel	A sensor can measure several quantities (multi-probe) and each unit is used by many sensors. m:n cannot be stored with a single FK, therefore an associative table with a composite PK.
sensors → sensor_unit_rel	1 : n	sensor_unit_rel.sensor_id	First half of the m:n resolution. ON DELETE CASCADE: if a sensor is removed, its configuration rows go with it.
measurement_units → sensor_unit_rel	1 : n	sensor_unit_rel.unit_id	Second half of the m:n resolution.
sensor_unit_rel → measurements	1 : n	measurements(sensor_id, unit_id)	Each reading belongs to one sensor *and* one unit; a sensor/unit pair produces many readings over time. Referencing the pair (not the sensor alone) guarantees the sensor really measures that quantity and tells you which unit a value has.

Why no 1:1 relationship? A 1:1 relation is used when attributes are optional or need to be separated (e.g. calibration details only some sensors have). Nothing in this data set needs that, so 1:1 is not used – a 1:1 split would not change the normal form.

10. Entity-Relationship Diagram (rotated – turn page)

Crow's-foot notation: **||** = exactly one, **o<** = zero or many, **|<** = one or many.



11. SQL DDL

```
-- 3NF schema for the pipe-network measurement data (SQLite / PostgreSQL compatible)
CREATE TABLE junctions (
    id          INTEGER PRIMARY KEY,
    code        TEXT    NOT NULL UNIQUE,      -- e.g. 'J-A'
    x_m         REAL    NOT NULL,
    y_m         REAL    NOT NULL
);

CREATE TABLE pipes (
    id               INTEGER PRIMARY KEY,
    code             TEXT    NOT NULL UNIQUE, -- e.g. 'P-01'
    material         TEXT    NOT NULL,
    diameter_mm      INTEGER NOT NULL,
    install_year     INTEGER NOT NULL,
    from_junction_id INTEGER NOT NULL REFERENCES junctions(id),
    to_junction_id   INTEGER NOT NULL REFERENCES junctions(id),
    CHECK (from_junction_id <> to_junction_id)
);

CREATE TABLE sensors (
    id          INTEGER PRIMARY KEY,
    code        TEXT    NOT NULL UNIQUE,      -- e.g. 'S-101'
    name        TEXT    NOT NULL,
    pipe_id     INTEGER NOT NULL REFERENCES pipes(id),
    position_m  REAL    NOT NULL              -- distance from the pipe's start junction
);

CREATE TABLE measurement_units (
    id          INTEGER PRIMARY KEY,
    symbol      TEXT    NOT NULL UNIQUE,      -- e.g. 'bar'
    name        TEXT    NOT NULL,
    quantity    TEXT    NOT NULL              -- e.g. 'pressure'
);

CREATE TABLE sensor_unit_rel (
    sensor_id   INTEGER NOT NULL REFERENCES sensors(id) ON DELETE CASCADE,
    unit_id     INTEGER NOT NULL REFERENCES measurement_units(id),
    PRIMARY KEY (sensor_id, unit_id)
);

CREATE TABLE measurements (
    id          INTEGER PRIMARY KEY,
    time        TIMESTAMP NOT NULL,
    sensor_id   INTEGER   NOT NULL,
    unit_id     INTEGER   NOT NULL,
    value       REAL      NOT NULL,
    FOREIGN KEY (sensor_id, unit_id) REFERENCES sensor_unit_rel(sensor_id, unit_id),
    UNIQUE (time, sensor_id, unit_id)
);
```

12. Check: Is the Decomposition Lossless?

A correct normalisation must not lose information: joining all tables must give back exactly the original flat file. The CSV was loaded into the schema (SQLite) and recombined with the following query – all 36 rows matched the original 1:1:

```
SELECT m.time, s.code, s.name, s.position_m,
       p.code, p.material, p.diameter_mm, p.install_year,
       jf.code, jf.x_m, jf.y_m, jt.code, jt.x_m, jt.y_m,
       u.quantity, u.symbol, u.name, m.value
FROM measurements m
JOIN sensors s            ON s.id  = m.sensor_id
JOIN pipes p              ON p.id  = s.pipe_id
JOIN junctions jf         ON jf.id = p.from_junction_id
JOIN junctions jt         ON jt.id = p.to_junction_id
JOIN measurement_units u  ON u.id  = m.unit_id;
```

Table	Rows after import
junctions	4
pipes	3
sensors	4
measurement_units	3
sensor_unit_rel	6
measurements	36

Instead of 36 × 16 = 576 values with heavy repetition, each fact is now stored exactly once. Test of the composite FK: inserting a *bar* value for flow meter S-102 is rejected with `FOREIGN KEY constraint failed`.

13. Common Mistakes (Checklist for Reviewing Student Diagrams)

- **measurements without unit_id** – if a sensor measures several quantities, a value of 4.11 cannot be interpreted. The unit belongs to the reading (or the sensor/unit pair).
- **FK in the wrong direction** – the FK sits on the n-side. `pipes.from_junction_id` → `junctions.id` is correct; a link from `pipes.id` to `junctions.id` (primary key to primary key) does not express “a pipe connects two junctions”.
- **Only one junction FK** – a pipe has two ends; one FK cannot model the network topology.
- **Storing the position as "x;y" text** – violates 1NF; split into two numeric columns.
- **Copying pipe attributes into sensors** – transitive dependency, violates 3NF.
- **m:n without associative table** – e.g. a column `unit_ids` with a list, or a `unit_id` in sensors (then a sensor can only have one unit).
- **Key of the association table** – should be the composite (sensor_id, unit_id) so the same pair cannot be stored twice.